

Parth Patel
CSCI 4104

11/27/23
Problem Set 6

34.1-1 Define the optimization problem LONGEST-PATH-LENGTH as the relation that associates each instance of an undirected graph and two vertices with the number of edges in a longest simple path between two vertices. Define the decision problem LONGEST-PATH = $\{ \langle G, u, v, k \rangle : G = (V, E) \text{ is an undirected graph, } u, v \in V, k \geq 0 \text{ is an integer, and there exists a simple path from } u \text{ to } v \text{ in } G \text{ consisting of at least } k \text{ edges} \}$. Show that the optimization problem LONGEST-PATH-LENGTH can be solved in polynomial time if and only if LONGEST-PATH $\in P$.

Proof:

Let LONGEST-PATH be a decision problem, since the question defines it to be so. Let's prove the statement from the question above via showing pseudocode for both the forwards and backwards case of the if-and-only-if statement.

Forwards Case: if the optimization problem LONGEST-PATH-LENGTH can be solved in polynomial time, then LONGEST-PATH can be solved in polynomial time as well.

Consider the pseudocode:

LONGEST-PATH (G : an undirected graph, where $G = (V, E)$, k : an integer, u : a vertex in G , v : a vertex in G):

Let "longest-path" be an integer value initially set to negative infinity.

longest-path = LONGEST-PATH-LENGTH(G, u, v)

if longest-path $\geq k$:

return true

else

return false

end function

(NEXT PAGE)

34.1-1 (cont.)

Proof: (cont.)

Forwards Case: (cont.)

Note how the part of LONGEST-PATH that has at least the greatest runtime in the algorithm is when the LONGEST-PATH-LENGTH function is called. I can assume this, because all of the other parts of LONGEST-PATH are constant time: initializing the "longest-path" integer variable, set "longest-path" to a new value, the if-else statements and return statements are all constant time algorithms. So, the worst case runtime of LONGEST-PATH is the same as LONGEST-PATH-LENGTH. We know that LONGEST-PATH-LENGTH has a polynomial runtime, so that means LONGEST-PATH also has a polynomial runtime. Therefore, the forwards case holds.

Backwards Case: IF LONGEST-PATH is a polynomial time decision problem, then the optimization problem LONGEST-PATH-LENGTH can be solved in polynomial time.

Consider the pseudocode:

LONGEST-PATH-LENGTH(G : an undirected graph, where $G = (V, E)$,
 u : a vertex in G , v : a vertex in G):

Let "num-vertices" be an integer value initialized to be 0.
num-vertices = $|V|$ // This is the size of V , total # of vertices
for $i = 0$ in num-vertices:

Let "longest-path" be an integer value initialized to be negative infinity.

if LONGEST-PATH(G, u, v, i) == true:

if $i > \text{longest-path}$
longest-path = i

return longest-path
end function.

(NEXT PAGE)

★ Note about pseudocode for Backwards Case: The reason I have the for loop run for $|V|$ iterations is because the max length of the longest path between u and v is the max number of vertices, since we are checking for simple paths. I then check to see if the longest path from u to v is at least size " i ". If " i " is greater than the current "longest-path", then I update "longest-path".

34.1-1 (cont.)

Proof: (cont.)

Backwards Case: (cont.)

Note how the part of LONGEST-PATH-LENGTH that has at least the greatest runtime is when the function call to LONGEST-PATH is made.

I can assume this, because initializing "num-vertices" and "longest-path" as integers was a constant time operation. The for-loop itself will run for $|V|$ iterations, where $|V|$ is the size of the set of vertices in the graph G . The nested inner if-statement in the for loop and changing the value of longest-path are also constant time operations. The outer if-statement makes a call to LONGEST-PATH to check if a path from vertex " u " to vertex " v " with length " i " exists, which takes some polynomial time.

So, by these assumptions, the worst possible runtime for LONGEST-PATH-LENGTH is polynomial time.

Thus, the Backwards Case holds.

Since both the forwards and backwards cases have been proven true, this means that the original statement has also been proven true. ■

34.2-6 A hamiltonian path in a graph is a simple path that visits every vertex exactly once. Show that language $\text{HAM-PATH} = \{ \langle G, u, v \rangle : \text{there is a hamiltonian path from } u \text{ to } v \text{ in graph } G \}$ belongs to NP.

Proof:

Consider the pseudocode:

$\text{HAM-PATH}(G: \text{a graph where } G = (V, E), u: \text{a source vertex in } G, v: \text{destination vertex in } G, p: \text{the list of vertices that form a hamiltonian path from } u \text{ to } v):$

Let "all-vertices" be a deep copy of the list of vertices V from graph G .

for n in p : // Note that the size of "all-vertices" is: $| \text{all-vertices} |$

// Let $\text{remove}(\text{list}, \text{element})$ use a linear search

// algorithm that runs in $O(| \text{all-vertices} |)$ and let "remove"

// return true if the element was successfully removed,

// and return false if the element was not found.

if $\text{remove}(\text{all-vertices}, n) == \text{false}$: // it means "n" has
return false // appeared before

// Let $\text{is-empty}(\text{list})$ be a $O(1)$ operation that checks if
// the list is empty. Returns true if empty, false otherwise.

if $\text{is-empty}(\text{all-vertices}) == \text{false}$: // it means not every element
return false // was contained in the ham. path "p"

end function

This pseudocode creates a deep copy of all vertices in the graph G (this operation takes $O(|V|)$ time, where $|V|$ is the size of V , which is the list of vertices in G). Then, it traverses the path called "p". For each vertex found, it is removed from "all-vertices". If the vertex isn't found in "all-vertices", that means it was removed already, implying that there is a duplicate vertex in "p", hence returning false. This for-loop runs in $O(| \text{all-vertices} | \cdot |p|)$, where $| \text{all-vertices} |$ is the size of "all-vertices" (a list) and $|p|$ is the size of "p" (a list representing the path). Lastly, the cost of the "is-empty" method and if-statement is constant time.

34.2-6 (cont.)

Proof: (cont.)

So, the different runtimes of the different parts of this pseudocode are: $O(|V|)$, $O(|\text{all-vertices}| \cdot |P|)$, and $O(1)$, which are all polynomial time. Thus, this verification algorithm of the hamiltonian path runs in polynomial time, which means the hamiltonian path decision question is verifiable in polynomial time, implying that it is in NP. 

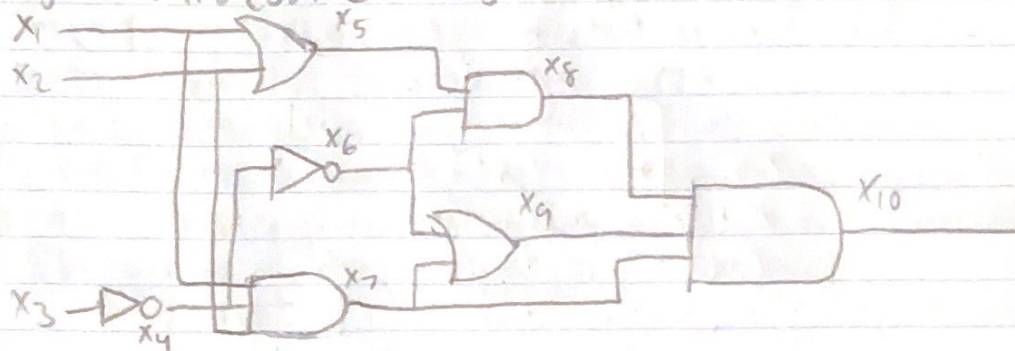
 or-gate

 and-gate

 inverter

34.3-1 Verify that the circuit in Figure 34.8 (b) is unsatisfiable.

Figure 34.8 (b): (I assigned x_4 to x_{10} based on the lecture slides)



Truth Table:

x_1	x_2	x_3	$x_4 \equiv \neg x_3$	$x_5 \equiv (x_1 \vee x_2)$	$x_6 \equiv x_3$	$x_7 \equiv (x_1 \wedge x_2 \wedge x_4)$	$x_8 \equiv (x_5 \wedge x_6)$	$x_9 \equiv (x_6 \vee x_7)$
1	1	1	0	1	1	0	1	1
1	1	0	1	1	0	1	0	1
1	0	1	0	1	1	0	1	1
1	0	0	1	1	0	0	0	0
0	1	1	0	1	1	0	1	1
0	1	0	1	1	0	0	0	0
0	0	1	0	0	1	0	0	1
0	0	0	1	0	0	0	0	0

$x_{10} \equiv (x_7 \wedge x_8 \wedge x_9)$

0
0
0
0
0
0
0
0

Since all truth values for x_{10} are zero, this means the circuit in Figure 34.8 (b) is unsatisfiable.

39.4-2 Show the 3-CNF formula that results upon using the method of Theorem 34.10 on the formula (34.3)

Given from the textbook:

$$\begin{aligned}\phi' = & y_1 \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2)) \\ & \wedge (y_2 \leftrightarrow (y_3 \vee y_4)) \\ & \wedge (y_3 \leftrightarrow (x_1 \rightarrow x_2)) \\ & \wedge (y_4 \leftrightarrow \neg y_5) \\ & \wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \\ & \wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3))\end{aligned}$$

$$\phi'' = (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee \neg x_2)$$

$$\textcircled{1} \begin{array}{ccc|c} y_1 & y_2 & x_2 & (y_1 \leftrightarrow (y_2 \wedge \neg x_2)) \\ \hline 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{array}$$

$$\textcircled{2} \begin{array}{ccc|c|ccc|c} y_2 & y_3 & y_4 & y_2 \leftrightarrow (y_3 \vee y_4) & y_3 & x_1 & x_2 & y_3 \leftrightarrow (x_1 \rightarrow x_2) \\ \hline 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{array}$$

$$\begin{array}{ccc|c|ccc|c|ccc|c} \textcircled{4} & y_4 & y_5 & y_4 \leftrightarrow \neg y_5 & \textcircled{5} & y_5 & y_6 & x_4 & y_5 \leftrightarrow (y_6 \vee x_4) & \textcircled{6} & y_6 & x_1 & x_3 & y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3) \\ \hline 1 & 1 & & 0 & 1 & 1 & 1 & & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & & 1 & 1 & 1 & 0 & & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & & 1 & 1 & 0 & 1 & & 1 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & & 0 & 1 & 0 & 0 & & 0 & 1 & 0 & 0 & 0 & 0 \\ & & & & 0 & 1 & 1 & & 0 & 1 & 1 & 1 & 1 & 1 \\ & & & & 0 & 1 & 0 & & 0 & 1 & 0 & 0 & 0 & 0 \\ & & & & 0 & 0 & 1 & & 0 & 0 & 1 & 1 & 0 & 0 \\ & & & & 0 & 0 & 0 & & 1 & 0 & 0 & 0 & 1 & 1 \end{array}$$

34.4-2 (cont.)

I numbered each of my truth tables ① to ⑥. I will refer to these numbers for my conversions.

Now, I will write literals and clauses for each truth value that equals zero, and negate them:

Table 1:	Negate:
$(y_1 \wedge y_2 \wedge x_2) \vee$	$\longrightarrow (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge$
$(y_1 \wedge \neg y_2 \wedge x_2) \vee$	$\longrightarrow (\neg y_1 \vee y_2 \vee \neg x_2) \wedge$
$(y_1 \wedge y_2 \wedge \neg x_2) \vee$	$\longrightarrow (\neg y_1 \vee y_2 \vee x_2) \wedge$
$(\neg y_1 \wedge y_2 \wedge \neg x_2)$	$\longrightarrow (y_1 \vee \neg y_2 \vee x_2)$

Table 2:	Negate:
$(y_2 \wedge \neg y_3 \wedge \neg y_4) \vee$	$\longrightarrow (\neg y_2 \vee y_3 \vee y_4) \wedge$
$(\neg y_2 \wedge y_3 \wedge y_4) \vee$	$\longrightarrow (y_2 \vee \neg y_3 \vee \neg y_4) \wedge$
$(y_2 \wedge y_3 \wedge \neg y_4) \vee$	$\longrightarrow (\neg y_2 \vee \neg y_3 \vee y_4) \wedge$
$(\neg y_2 \wedge \neg y_3 \wedge y_4)$	$\longrightarrow (y_2 \vee y_3 \vee \neg y_4)$

Table 3:	Negate:
$(y_3 \wedge x_1 \wedge \neg x_2) \vee$	$\longrightarrow (\neg y_3 \vee \neg x_1 \vee x_2) \wedge$
$(\neg y_3 \wedge x_1 \wedge x_2) \vee$	$\longrightarrow (y_3 \vee \neg x_1 \vee \neg x_2) \wedge$
$(y_3 \wedge \neg x_1 \wedge x_2) \vee$	$\longrightarrow (\neg y_3 \vee x_1 \vee \neg x_2) \wedge$
$(\neg y_3 \wedge \neg x_1 \wedge \neg x_2)$	$\longrightarrow (y_3 \vee x_1 \vee x_2)$

Table 4:	Negate:
$(y_4 \wedge y_5) \vee$	$\longrightarrow (\neg y_4 \vee \neg y_5) \wedge$
$(\neg y_4 \wedge \neg y_5)$	$\longrightarrow (y_4 \vee y_5)$

Table 5:	Negate:
$(y_5 \wedge \neg y_6 \wedge \neg x_4) \vee$	$\longrightarrow (\neg y_5 \vee y_6 \vee x_4) \wedge$
$(\neg y_5 \wedge y_6 \wedge x_4) \vee$	$\longrightarrow (y_5 \vee \neg y_6 \vee \neg x_4) \wedge$
$(y_5 \wedge y_6 \wedge \neg x_4) \vee$	$\longrightarrow (\neg y_5 \vee \neg y_6 \vee x_4) \wedge$
$(\neg y_5 \wedge \neg y_6 \wedge x_4)$	$\longrightarrow (y_5 \vee y_6 \vee \neg x_4)$

Table 6:	Negate:
$(y_6 \wedge x_1 \wedge x_3) \vee$	$\longrightarrow (\neg y_6 \vee \neg x_1 \vee \neg x_3) \wedge$
$(y_6 \wedge \neg x_1 \wedge \neg x_3) \vee$	$\longrightarrow (\neg y_6 \vee x_1 \vee x_3) \wedge$
$(\neg y_6 \wedge x_1 \wedge \neg x_3) \vee$	$\longrightarrow (y_6 \vee \neg x_1 \vee x_3) \wedge$
$(\neg y_6 \wedge \neg x_1 \wedge x_3)$	$\longrightarrow (y_6 \vee x_1 \vee \neg x_3)$

34.4-2 (cont.)

Let's join all of the negated clauses together from the previous page with conjunctions, so that:

$$\begin{aligned}\Phi'' = & y_1 \wedge (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee \neg y_2 \vee x_2) \\ & \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2) \\ & \wedge (\neg y_2 \vee y_3 \vee y_4) \wedge (y_2 \vee \neg y_3 \vee \neg y_4) \\ & \wedge (y_2 \vee \neg y_3 \vee y_4) \wedge (y_2 \vee y_3 \vee \neg y_4) \\ & \wedge (\neg y_3 \vee \neg x_1 \vee x_2) \wedge (y_3 \vee \neg x_1 \vee \neg x_2) \\ & \wedge (y_3 \vee x_1 \vee \neg x_2) \wedge (y_3 \vee x_1 \vee x_2) \\ & \wedge (\neg y_4 \vee \neg y_5) \wedge (y_4 \vee y_5) \\ & \wedge (\neg y_5 \vee y_6 \vee x_4) \wedge (y_5 \vee \neg y_6 \vee \neg x_4) \\ & \wedge (y_5 \vee \neg y_6 \vee x_4) \wedge (y_5 \vee y_6 \vee \neg x_4) \\ & \wedge (\neg y_6 \vee \neg x_1 \vee \neg x_3) \wedge (\neg y_6 \vee x_1 \vee x_3) \\ & \wedge (y_6 \vee \neg x_1 \vee x_3) \wedge (y_6 \vee x_1 \vee \neg x_3)\end{aligned}$$

Now, let's add the auxiliary variables, p and q , in order to make the terms that have only 1 or 2 literals have 3, so that way 3-CNF can be achieved:

$$\begin{aligned}\Phi''' = & (y_1 \vee p \vee q) \wedge (y_1 \vee \neg p \vee q) \wedge (y_1 \vee \neg p \vee \neg q) \wedge (y_1 \vee p \vee \neg q) \\ & \wedge (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee \neg y_2 \vee x_2) \\ & \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2) \\ & \wedge (\neg y_2 \vee y_3 \vee y_4) \wedge (y_2 \vee \neg y_3 \vee \neg y_4) \\ & \wedge (y_2 \vee \neg y_3 \vee y_4) \wedge (y_2 \vee y_3 \vee \neg y_4) \\ & \wedge (\neg y_3 \vee \neg x_1 \vee x_2) \wedge (y_3 \vee \neg x_1 \vee \neg x_2) \\ & \wedge (y_3 \vee x_1 \vee \neg x_2) \wedge (y_3 \vee x_1 \vee x_2) \\ & \wedge (\neg y_4 \vee \neg y_5 \vee p) \wedge (\neg y_4 \vee \neg y_5 \vee \neg p) \\ & \wedge (y_4 \vee y_5 \vee p) \wedge (y_4 \vee y_5 \vee \neg p) \\ & \wedge (\neg y_5 \vee y_6 \vee x_4) \wedge (y_5 \vee \neg y_6 \vee \neg x_4) \\ & \wedge (y_5 \vee \neg y_6 \vee x_4) \wedge (y_5 \vee y_6 \vee \neg x_4) \\ & \wedge (\neg y_6 \vee \neg x_1 \vee \neg x_3) \wedge (\neg y_6 \vee x_1 \vee x_3) \\ & \wedge (y_6 \vee \neg x_1 \vee x_3) \wedge (y_6 \vee x_1 \vee \neg x_3)\end{aligned}$$

Since all clauses are in 3-CNF, and all clauses are connected by conjunctions, so this has been proven to be 3-CNF.

34.1-5 Show that if an algorithm makes at most a constant number of calls to polynomial-time subroutines and performs an additional amount of work that also takes polynomial time, then it runs in polynomial time.

Let's assume that the worst case runtime of this algorithm is the runtime of the polynomial-time subroutines that are repeated a constant number of times plus the runtime of that additional work that is also in polynomial time.

So, let's define the time complexities of these subroutines as:

$O(n^{t_1}), O(n^{t_2}), \dots, O(n^{t_k})$. Let's also define the time complexity of the additional work as: $O(n^a)$. Note that I can define these time complexities as "n" raised to some power, because the question says that these subroutines are all in polynomial time. The scalars $t_1, t_2, \dots, t_k$ represent the constant number of times that each subroutine is repeated. The scalar "a" is the exponent for the time of additional work. Since we do not know which subroutine has the slowest runtime, let's let $t = \max(t_1, t_2, \dots, t_k)$, where max finds the greatest number, which will be used to represent the slowest runtime.

So, now we can write the time complexity of this algorithm as: $O(n^t + n^a)$, and this is still a polynomial-time algorithm, since the addition of 2 polynomial terms still equals a polynomial.